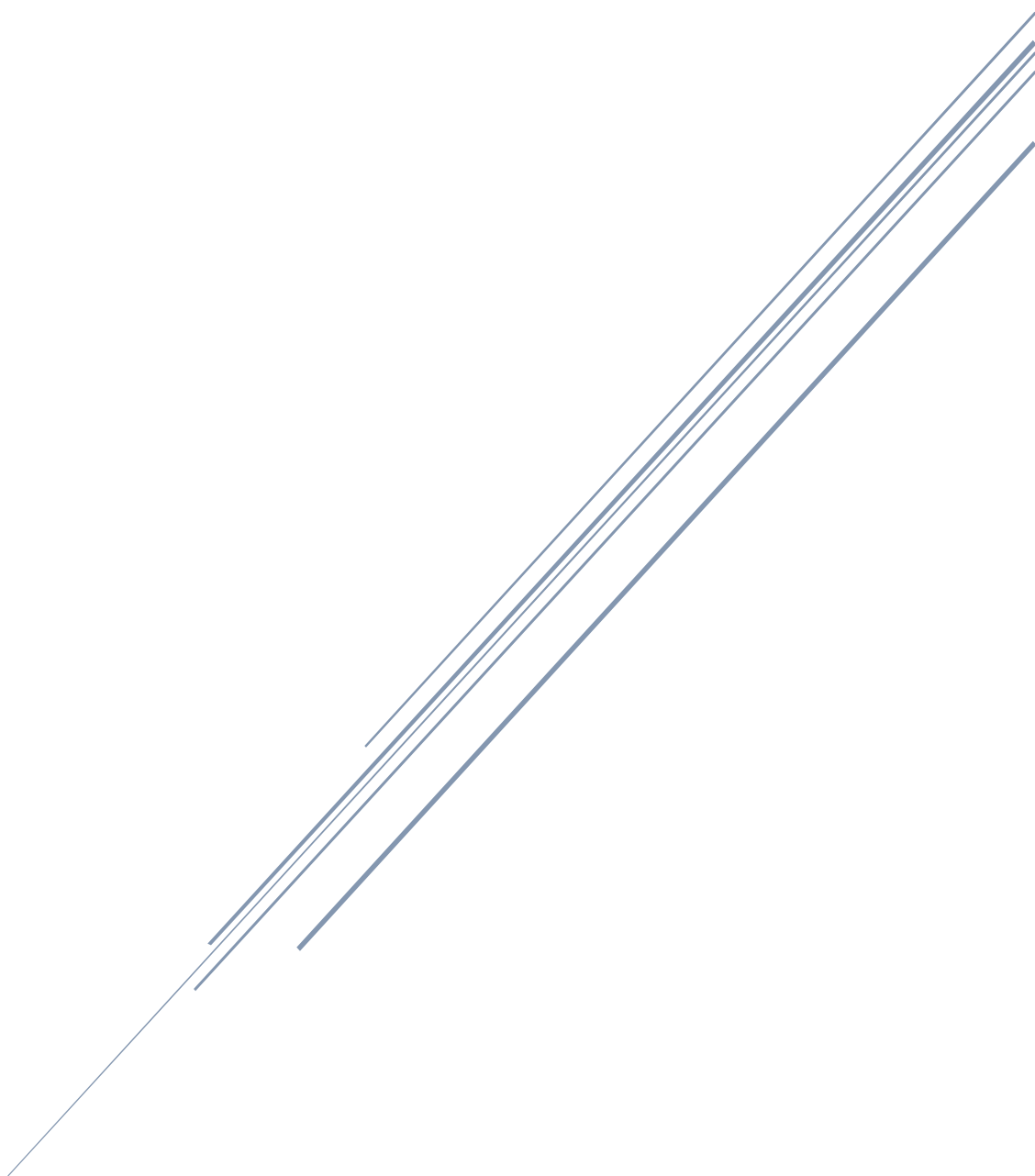


プログラムポートフォリオ



目次

自己紹介	2
学習した技術	2
ゲームプレイ履歴	3
Tech Stadium について	5
作品・学習内容紹介	6
2D 横スクロールアクションゲーム	6
Unity 公式チュートリアル『Tank』の改変	1 3
終わりに	1 5

自己紹介

氏名：（氏名）

学習した技術

種類	期間	内容・作品
Unity・C#	2 か月 (2019/6～7)	・ チュートリアル参照して、ゲームの作成 ポートフォリオ内に記載
CakePHP・phpMyAdmin	2 か月 (2019/6～7)	・ 上記 Unity ゲームにおける API の作成
UnrealEngine4 C++	1 か月 (2019/5)	・ ブロック崩しゲームを作成 BluePrint メインで作成。C++は入門書 1 冊を完了。
Creative Cloud CS5 DreamWeaver, PhotoShop, FLASH (AS3)	半年程度 (2012/7～2013/1)	フリーランス ・ デバッガー、タイムラインアニメーション作成 Web アプリ、ブラウザゲーム等 ・ PhotoShop 素材データ圧縮等 ・ 小規模コーポレートサイト作成

ゲームプレイ履歴（特に関心があったものを抜粋）

【スマホゲーム】

スマホゲームは、タップするとすぐに起動する、手軽にどこでもゲームできるので、ここ数年はスマホゲームをメインにプレイしています。スマホではクラウドバックアップを取っておけば、携帯がなくなってもデータが保持されるという安心感もメリットだと感じています。

・『ドラゴンクエスト 4,5,6,7,8,モンスターズ』（iOS / SQUARE ENIX）コンシューマからiOS移植されているものは全て全クリしました。魅力的なキャラデザイン（味方/敵）、世界観、職業システム、カジノやすごろく等のミニゲーム、ストーリー性、魔法や特技などの種類の豊富さとゲームバランスの絶妙さ、どれを取っても好きです。4でボス敵だったデスピサロが仲間になる、5で結婚相手の選択により仲間と子供が変わる、8でキーファが突然仲間から抜ける等の衝撃的な展開があるのも好きです。

・『アーチャー伝説』（iOS / HABBY）現在 5.失われた城。シューティングです。序盤は簡単にクリアしていただけますが、途中 Stage3 くらいから異様に難しくなっていきます。このゲームの特徴は操作がこれ以上無いほどシンプルで、無操作時に球発射、上下左右移動のみです。ただそれだけの操作なのですが、逆にシンプルな分、敵にこちらの攻撃を当てて倒す、球を避けるということのみに専念出来るので、没頭していきます。拡張アイテムをルーレットで引く運要素が入っているのも何度かトライしていると結果が良くなったりするので博打的なハマリ要素だと感じます。

・『Plague Inc. -伝染病株式会社-』（iOS / Ndemic Creations）

伝染病を広めていき、世界中の人を殺したらクリアになるというクレイジーなコンセプトのゲームです。

しかし、ゲーム内容は論理的かつ合理的です。渡り鳥の移動やオリンピック開催などによって、伝染の仕方が変わります。また、陸、空、海上の輸送手段によって感染したり、動物感染したり、気候の温暖、湿度の差など多数のパラメータに値を振り分けることによって伝染の仕方を変えていくという妙にリアルな疑似的ウイルス伝搬体験ができます。地球人口を全滅したときには妙な達成感があります。

同社が開発した新作 Rebel Inc は Plague と似ているところがありますが、政治的、軍略的に地域を統治（洗脳）するというゲームでそちらを現在プレイしています。

【コンソールゲーム】

ファミコンからプレイステーション 2 の世代まで集中的にプレイしていました。

名作揃いですが、厳選した特に好きなタイトルと感想は下記の通りです。

ファミコン：くにおくん運動会、スーパーマリオブラザーズ

くにおくんシリーズには時代劇やバスケ、ドッジボール等名作揃いですが、中でも運動会の競技のバリエーションと操作の柔軟性（ハードルの板をジャンプして後ろの敵に投げる、プールで足を引っ張り溺れさせる、柔道の技の種類の豊富さ等）がファミコン時代とは思えない程に完成度が高く感じました。

スーパーマリオブラザーズは言わずと知れた全てのゲーム人気を牽引してきた存在だと思っています。

未だに耳に残っているメロディーや効果音、ドッスンやクッパ等が鮮明に思い出されます。

当時初期のファミコンでは 40kB しかデータ容量（軽くした JPEG 画像 1 枚と同程度）が無かったらしく、そこに画像や音楽、システムを詰め込んでいたということなので、驚愕します。昔のゲームはアイデアが沢山あっても、容量の問題で厳選されたアイデアしか採用出来なかったらしいです。だから単純だけど、面白いゲームが多かったのではないかと思います。ファミコンのゲームは今でも耳に残る音楽が多い気がします。

スーパーファミコン：ドカポン 4、ミリティア

ドカポンは 123 が有名な気がしますが、私は 4 が好きです。4 人プレイできるから 4 という意味で実は 4 がドカポンシリーズの初代だと最近知りました。対戦方法が他に無い独特の読みと駆け引きを生み、レベルアップ時 HP や攻撃などのステータスに値を割り振りが出来るシステムもオリジナルなキャラを作れる楽しさがありました。特に友達と 4 人でやると楽しかったです。最下位が悪魔に魂を売るという逆転設定もあったので、負けていても最後まで楽しめました。

ミリティアは軍事シミュレーションゲームで、ロボットを敵地に送り込んだり、燃料補給基地を作ったり、迎撃ミサイルを設置したりと多彩な戦争シミュレーションが出来ました。

プレイステーション：モンスターファーム、パラッパラッパー

モンスターファームは CD でモンスターを生み出すという新しいシステムで、対戦方法等はポケモンのようにドラゴン等のレアキャラを作れた時うれしかったです。。パラッパラッパーは音ゲーですが、世界観は西欧のアニメのような雰囲気、音楽もヒップホップ系で統一感があり好きでした。

プレイステーション 2：グランツーリスモ 3、Final Fantasy X、鉄拳 5 (+アーケード)

PS2 になるとグラフィックの美しさが格段に上がり、FFX は当時、映画を見ているような感覚になりました。グランツーリスモも当時、実物と見分けがつかない程、高精度の映像だと思いました。

鉄拳はアーケードでもやっていましたが、家庭でやれるということが贅沢な感覚で楽しかったです。

Tech Stadium について

株式会社クリーク・アンド・リバー社運営の Tech Stadium Unity コース第 10 期を受講しました。

期間：2019年6月17日（月）～2019年7月29日（月）

カリキュラム：

1. クライアント技術学習(unity)

- ・ ゲーム業界と技術の概要
- ・ Git の差分情報管理
- ・ C#プログラミング（Break Point Debug、Scene 間の変数参照、instance の自動生成等）
- ・ 位置座標取得と移動、AddForce、Rigidbody 等の数学/物理的数値の取扱い
- ・ 画面遷移
- ・ Timer 表示
- ・ UI (uGUI を利用した UI 作成)
- ・ Camera 制御（相対座標固定や画角固定等）
- ・ Audio 制御(Mixer を使った音源の重ね方、Audio Source の clip を script から変更等)
- ・ ParticleSystem
- ・ 接触判定(Collision、Trigger)

2. サーバー技術学習(CakePHP)

- ・ ゲームサーバー基礎/サーバー構成
- ・ LAMP 環境構築(php,mysql)
- ・ Debug の有効な使い方
- ・ フレームワーク利用(CakePHP を利用した MVC、ORM)
- ・ DB 基礎(データベース/テーブル作成、query 制御)
- ・ Web ページ作成
- ・ API 学習/作成（json 取得表示、ARC や Postman から POST レコード挿入テスト、DB 側確認）

3. クライアント、サーバー連携技術学習(unity & CakePHP)

- ・ Unity でのデータアクセス/ネットワークアクセス
- ・ CakePHP での API 作成、Unity との繋ぎこみ
- ・ WebGL による Build で html ファイルを生成し、レンタルサーバへFTPでアップロード

作品・学習内容紹介

ポートフォリオ動画

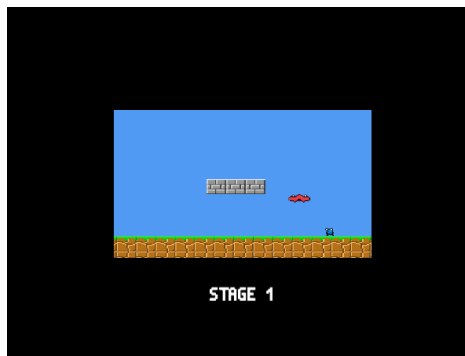
(プレイ youtube へアップロードしリンクを記載する。)

Git リポジトリ

(プロジェクト一式を gitlab へアップロードしリンクを記載する。)

2D 横スクロールアクションゲーム Rario

ゲームの画像



(左上：タイトル画面。クリックすると Stage1 表示画面に遷移。タイトル用の音楽が流れる。)

(右上：Stage 表示画面。2 秒後、自動でプレイ画面へ画面遷移)



(左上：プレイ画面。HP スライダーとタイム、スコアがあり、左 Ctrl キー押下で球を発射。Space キー押下でジャンプ。)

(右上：ゲームオーバー。地面が無い所に落ちるか HP が無くなるとゲームオーバー。)



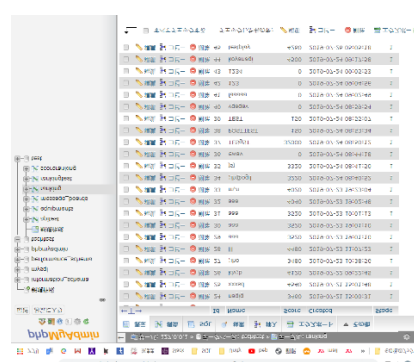
(左上：クリア。クリアすると自動で右に走っていき、結果画面に自動で移動。クリア音楽が流れる。)

(右上：結果画面。スコアとタイムに応じたボーナススコア、それらの合計スコアを表示。)

プレイヤー名を入力して決定をクリックすると、ランキング画面に遷移。)

ランキング

じゅんい	なまえ	スコア
1	TESgST	32000
2	カタカナ	19000
3	ひらがな	10000
4	xxxxsd	4540
5	koyanagi	4500



(左上；ランキング画面。PHP でソートして件数を指定して json データ取得、全数表示。)

(右上：SQL データベース画面。testplay のレコードが追加されている。)

ゲーム概要

1 基本説明

ユニティちゃんをプレイヤーとして使用しています。

敵を弾丸で打って倒しながら進み、落下したり、一定以上のダメージを受けると GameOver となります。

敵を倒した時のスコアとゲームクリア時のタイムに応じたスコアを足した値が最終スコアとなります。

ゴール時にプレイヤー名を入力してこの最終スコアをデータベース上のランキングに登録して、そのランキング結果を表示します。

馴染みのある操作感とシンプルなゲーム性で手軽に遊べるゲームです。また URL を入力するだけで Web 上でも遊べます。 <http://koyanagir.php.xdomain.jp/Rario/HTML/index.html>

2 操作説明

移動：↑→↓←キー

ジャンプ：space キー

ショット：左 Ctrl キー

3 制作環境

Unity 2019.1.4f1

XAMPP ver7.2.14 / PHP ver7.2.14 /MySQL ver15.1

CakePHP 3, phpMyAdmin

制作期間

2019 年 7 月 9 日（火）～2019 年 7 月 23 日（火）の 14 日間

技術面

全体概要

1. タイトル画面からランキング画面までの一連の画面遷移
2. スプライト画像を切り替えてキャラクターアニメーションを作成
3. タイムとスコア機能と Result 画面におけるシーン間での変数取得表示
4. BGM と効果音をタイトル〜クリア時まで配置、効果音はそれぞれのオブジェクトに設置して、バックミュージックは AudioManager にて一元管理
5. Result 画面にてプレイヤー名を入力した際に、サーバーに設置した API に POST 通信を行う事でデータベースにレコードを追加。
6. Ranking 画面にてサーバから GET 通信にてランキングデータ取得（XAMPP 環境）
7. WebGL の Build によって、HTML を作成して、FTP によってレンタルサーバ上にアップ（SQL データベース通信は出来ていない）<http://koyanagir.php.xdomain.jp/Rario/HTML/index.html>

技術面で特にこだわった部分

1. BGM と効果音をタイトル〜クリア時まで配置

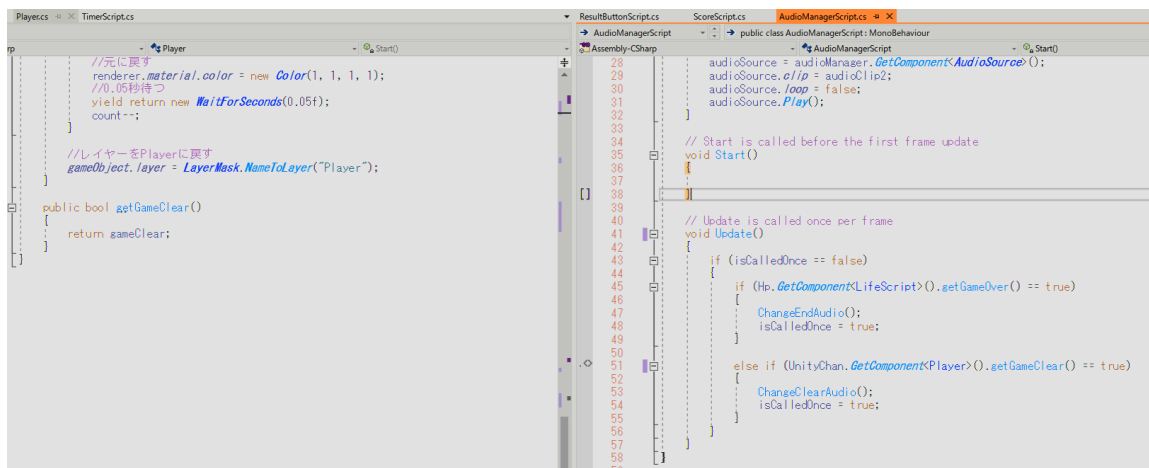
[苦労した点]

最初 Player の Component にバックミュージックをつける等して、メンテナンス性が悪い構造にしようとしていた。

[改善点]

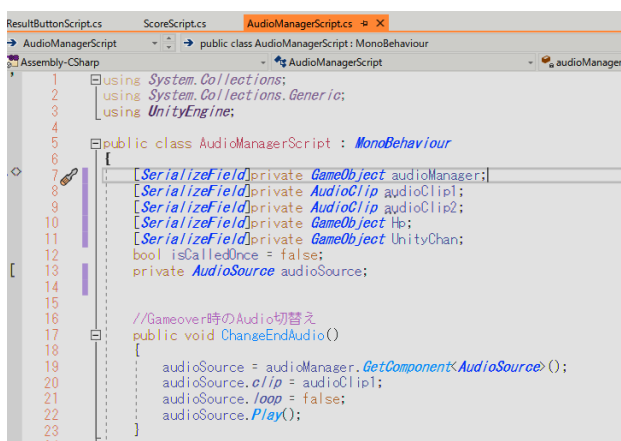
効果音はそれぞれのオブジェクトの Component に設定して、バックミュージックは空 Object として AudioManager にて一元管理するようにスクリプトを書き換えた。

具体的には GameClear や GameOver 等のブールフラグを取得して AudioManager によってミュージック切替えている。他に、変数は出来る限り public を使わないようにして、ゲッターや[SerializeField]private などによってカプセル化を意識した。



上記 getGameClear()で private bool gameClear 変数を AudioManager から呼び出すようにした。

下記は変数の定義を public ではなく、[SerializeField]private にしているカプセル化の例。



2. Ranking 画面の作成

[苦労した点]

- ① json データを GET 通信で取得出来てもデータ型が異なり、表示出来ない等の問題が発生した。
- ② 取得したランキングデータを表示する際に、単に一つのテキストに foreach でランキングデータを流し込んでいた為、レイアウトが整えられず、作り直しが必要となった。 下図 左：改善前 右：改善後

ランキング

じゅんい	なまえ	スコア
1	TestPlayer2000	
2	testman3900	
3	POSTman1500	
4	aaaaaaaaaaaaaaaa1400	

ランキング

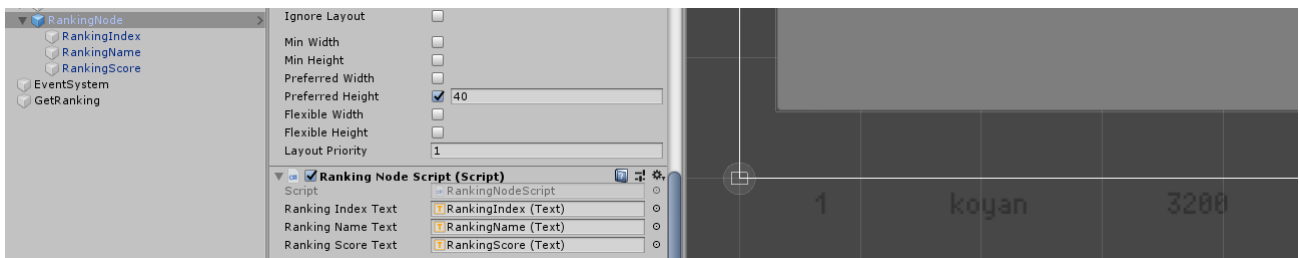
じゅんい	なまえ	スコア
1	TESgST	32000
2	カタカナ	19000
3	ひらがな	10000
4	xxxxsd	4540
5	koyanagi	4500

[改善点]

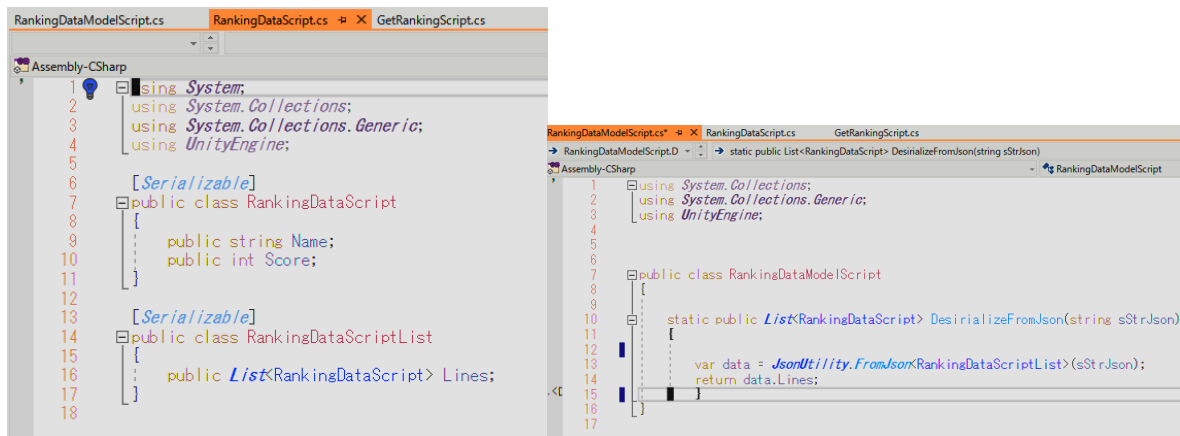
XAMPP 環境にて仮定の Local データベース（MySQL）から GET 通信にてランキングデータを取得。

```
RankingController.php
C:\Users\array\Documents\ts-un-stu_201906\20_ServerProject\htdocs\RarioAPI\src\Controller
106
107 public function getRankings()
108 {
109     $this->autoRender = false;
110
111     //テーブルからランキングリストをとってくる
112     $query = $this->Ranking->find("all");
113
114     //クエリー処理を行う。
115     $query->where(['Stage'=>1])>order(['Score'=>'DESC']); //降順
116     $query->limit(15); //取得件数を絞る
117
118     //jsonにシリアライズする。
119     $json = json_encode($query);
120
121     //jsonデータを返す。(レスポンスとして表示する。)
122     echo "{\n\"Lines\": \" . $json .\"}";
123 }
```

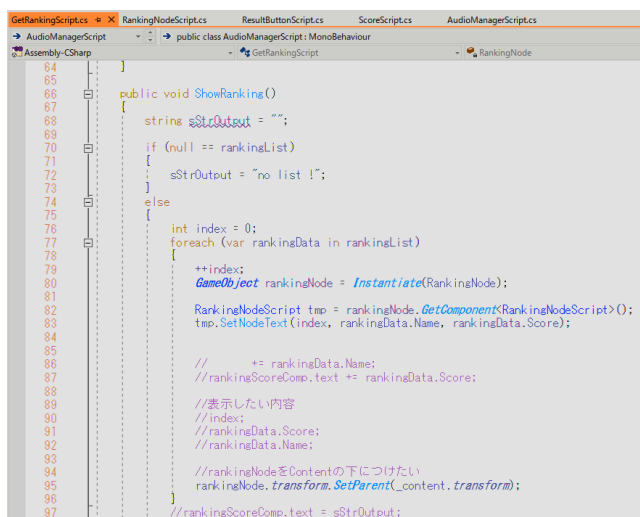
上記 CakePHP の Controller にてまず \$query をデータベースのテーブルの全件データを見つけるというクエリとして定義。そこから where にて Stage1 を条件指定、order にて Score を降順（DESC）に並び替え、limit(15)にて取得件数を 15 件に限定。



上図のように 1 行ずつのデータ（レコード）を RankingNode として定義してプレハブ化した。



Dictionary 型にパースしながら foreach で格納する miniJSON ではなく、JsonUtility にて List 型でデータ取得。これにより、データ型変換の必要がない為、コーディングがシンプルになり、取得時と取得してからのデータの取扱いが容易になった。（上図）



取得した List 型のデータはレコード単位をノードとしてプレハブ化して、foreach で自動生成している。

スライダーをつけており、件数が php 側の操作で増減しても、自動で対応できる。（上図）

Unity 公式チュートリアル『Tank』の改変

画像



(左図：通常時のショットの様子)



(右図：レーザーアイテム取得後のショットの様子)

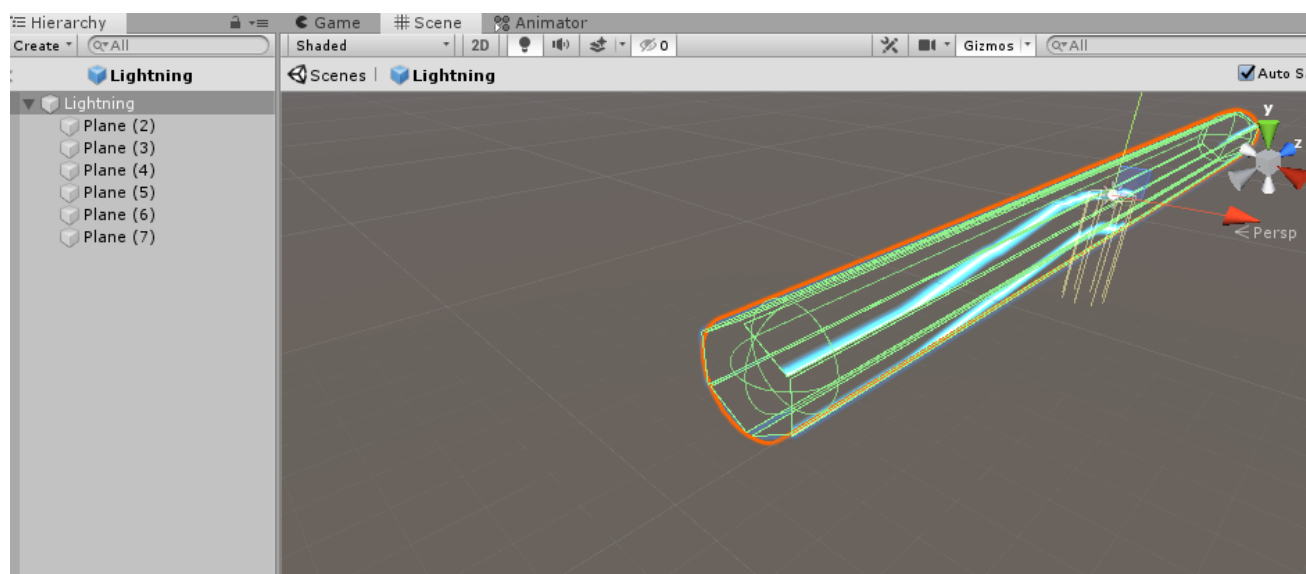
概要

Unity 公式チュートリアルに従って Tank プロジェクトを作成し、レーザー攻撃への切替機能を実装した。

2種類のアイテムを配置しており、通常の砲弾からレーザーに変わるアイテムとレーザーから通常砲弾に変わるアイテムを設置した。

技術面

初めビームを覆うように円柱の Collider を配置しようとしたが、角度によって平たく見えたりと、見え方が変わってしまったので、下図のように細長い板を六枚使って六面体を構成し、それぞれテクスチャの様に、どの面からも見え方が均一になるようにした。Shader は公式 Asset Store からダウンロードしたが、太さや見え方のスクリプトも多少修正した。Collider は円柱の空オブジェクトとしている。



```

using UnityEngine;
using UnityEngine.UI;

public class TankShooting : MonoBehaviour
{
    public enum WeaponType
    {
        shot, lightning
    };
}

```

```

using System.Collections;
using UnityEngine;

public class ItemScript : MonoBehaviour
{
    public TankShooting.WeaponType m_WeaponType;

    private void OnTriggerEnter(Collider other)
    {
        Destroy(gameObject);
    }
}

```

砲弾発射の部分で列挙型 enum で shot と lightning の 2 種類を定義。それをアイテムのスク립トで読み込み、インスペクタでアイテム毎に指定どちらのアイテムかを指定した。列挙型にしたのは、アイテムの種類が他に増えた場合に拡張性を持たせる目的。

```

public Slider m_AimSlider;

public class TankShooting
{
    public void Shoot()
    {
        if (m_WeaponType == WeaponType.shot)
        {
            Rigidbody shellInstance = Instantiate(m_Shell, m_FireTransform.position, m_FireTransform.rotation);
            shellInstance.velocity = m_CurrentLaunchForce * m_FireTransform.forward;
            m_ShootingAudio.clip = m_FireClip;
            m_ShootingAudio.Play();
            m_CurrentLaunchForce = m_MinLaunchForce;
        }
        else if (m_WeaponType == WeaponType.lightning)
        {
            GameObject lightningInstance = Instantiate(m_Lightning, m_LightningTransform.position, m_LightningTransform.rotation);
            float timer = 1.5f;
            Destroy(lightningInstance.gameObject, timer);
        }
    }
}

```

WeaponType の種類によって、プレハブからそれぞれの種類の砲弾を自動生成。

制作期間

2019年6月後半の5日間ほど

終わりに

Tech Studium の Unity コースにて初めて本格的にゲーム業界の基本技術に触れることが出来たと思います。

本コース内容では講義開始の週の平日から課題が度々あり、付いていくのに必死でした。しかし、終わってみると、そのように集中的に学ばせて頂いたお陰で、C#のプログラミングの基礎や、CakePHP、SQL 等のサーバー技術、Git、json 等のゲーム業界における基礎技術を習得しました。

まだ始まったばかりなので、勉強することは多々ありますが、積極的にどん欲に知識や技術を身に付けて、ゲーム業界のプログラマーとしてのキャリアを築いていきたいと願います。

以上